Introduction

The **ATM Management System** is a simple Java-based application designed to demonstrate basic concepts of **Object-Oriented Programming, classes, objects, ArrayList, loops, conditional statements, and user input**. The system allows users to create and store multiple accounts, view account details, and increase an employee's salary based on their name. A simple menu-driven interface provides four options: **Create Account, Display All Accounts, Raise Salary, and Exit**. The project demonstrates how Java can be used to build a basic interactive management system while organizing data efficiently using objects and an `ArrayList`.

```
===== ATM Menu =====
1. Create Account
2. Display Account
3. Raise Salary
4. Exit
Enter your choice: 1
Enter your name: vinod
Enter your age (19-59): 33
Enter designation (Programmer / Manager / Tester): Tester
Do you want to re-enter designation? (y/n): n
```

Create Account Feature

The **Create Account** feature allows the user to create and store multiple accounts using an `ArrayList`. The program takes the user's **name, age, and designation** as input. Age is validated to ensure it is between **19 and 59**. The user can select **Programmer, Manager, or Tester**, and the salary is automatically assigned based on the designation. Finally, an `Account` object is created using the entered details and added to the `ArrayList`, allowing multiple accounts to be stored and displayed later.

```
===== ATM Menu =====
1. Create Account
2. Display Account
3. Raise Salary
4. Exit
Enter your choice: 2

--- Account Details ---
Name       : vinod
Age        : 33
Designation: Tester
Salary     : 15000.00
-----------------------
```

Display All Accounts Feature

The **Display All Accounts** feature is used to show all the accounts stored in the `ArrayList`. First, the program checks whether the `ArrayList` is empty. If there are no accounts, it displays **"No accounts found."** Otherwise, a `for` loop goes through each account using `accounts.get(i)` and displays the **account number, name, age, designation, and salary**. Since the accounts are stored in an `ArrayList`, the feature can display **multiple accounts** that were created by the user.

```
===== ATM Menu =====
1. Create Account
2. Display Account
3. Raise Salary
4. Exit
Enter your choice: 3
Enter salary increase percentage (1-10): 6
Do you want to apply this salary increase? (y/n): y
Salary increased successfully!

===== ATM Menu =====
1. Create Account
2. Display Account
3. Raise Salary
4. Exit
Enter your choice: 2

--- Account Details ---
Name       : vinod
Age        : 33
Designation: Tester
Salary     : 15900.00
-----------------------
```

Raise Salary Feature

The **Raise Salary** feature allows the user to increase the salary of a particular employee by entering their **name**. The program first checks whether any accounts exist. It then searches the `ArrayList` using a `for` loop and `equalsIgnoreCase()` to find the employee by name. Once the account is found, the user enters a salary increase percentage between **1 and 10**. The program calculates and displays the **old salary and new salary**, then asks the user for confirmation. If the user enters `y`, the new salary is updated in the `Account` object; if `n` is entered, the change is cancelled.

Exit Feature

The **Exit** feature allows the user to safely close the ATM program. When the user selects **option 4**, the program displays a thank-you message, closes the `Scanner` using `sc.close()`, and uses `return` to stop the `main()` method and terminate the program. This prevents the program from continuing to display the ATM menu after the user has chosen to exit.

```
===== ATM Menu =====
1. Create Account
2. Display Account
3. Raise Salary
4. Exit
Enter your choice: 4
Thank you for using the ATM system!
```

Conclusion

The **ATM Management System** successfully demonstrates the use of Java programming concepts to create a simple and interactive application. The project uses **classes and objects** to represent accounts and an **ArrayList** to store multiple account details. It also demonstrates **loops, conditional statements, input validation, searching, and basic salary calculations**. Through the four main features—**Create Account, Display All Accounts, Raise Salary, and Exit**—the project provides a clear understanding of how Java concepts can be combined to develop a practical menu-driven program.